

Project 2 - report

2022036208 인범진

Code Analysis

- xv6에 kernel-level thread를 도입하기 위해 스켈레톤 코드가 기존 xv6 코드와 비교해서 무엇이 추가/변경되었고, 왜 그러한 변경이 필요한가를 다음의 세가지 부분으로 나누어 분석했다
 - `mm_struct` 도입과 thread-aware `struct proc`
 - `sscratch` 기반의 per-thread trapframe 메커니즘
 - `fork()` 를 `clone()` 으로 통합하고 thread 라이브러리를 추가한 user/syscall 계층
- 이 분석은 제공된 `git diff`를 참고했다

1. Introduce `mm_struct` and Thread-aware `struct proc`

1.1 변경의 동기 분석

- 원본 xv6에서 `struct proc`은 실행 단위(execution unit)와 주소 공간(address space)을 하나의 구조체에 묶었다
- `context`, `kstack`, `trapframe`이 실행 상태를 나타내며, `pagetable`, `sz`와 같은 메모리 상태가 같은 구조체 안에 들어 있어, 두 `proc`이 동일한 주소 공간을 공유할 수 없었다
 - 새로운 execution unit을 만들어야, 새로운 address space를 만들 수 있었다
- 스켈레톤 코드는 Linux의 방식을 활용하여 thread를 구현했다
- `task_struct` (우리 코드에선 `struct proc`) `mm_struct`를 포인터로 참조하게 하고, `CLONE_VM` 플래그 유무에 따라 `mm`을 공유할지(shared), 새로 만들지(copy)를 결정한다
 - 이때 copy의 방식은 기존의 fork와 같다

1.2 `kernel/proc.h` - 새 자료구조

```
struct mm_struct {
    pagetable_t pagetable;    // page table root
    uint64 sz;                // user memory size (bytes)
    int refcount;             // number of tasks sharing this mm
    struct spinlock lock;
};

#define CLONE_VM 0x0100      // share address space (thread)
```

- `mm_struct`에는 공유 대상이 되는 자원들이 있다
 - `pagetable`과 `sz`는 같은 thread group 안의 모든 thread가 동일한 값을 가져야 한다
 - `refcount`는 “마지막에 나가는 thread가 해제한다”는 규칙을 구현하기 위한 카운터로 판단했다
 - `lock`은 `sz` 증감이 동시에 일어날 때 메모리 상태가 일관되게 유지되도록 하기 위한 동기화를 위한 것이다
- 같은 헤더에서 `struct proc`은 다음과 같이 다시 설계되었다

```
struct proc {
    // ... 기존 lock, state, chan, pid, parent ...

    uint64 kstack;           // per-task (그대로)
    struct mm_struct *mm;     // pagetable + sz → mm 포인터로 교체
    struct trapframe *trapframe; // per-task
    struct context context;   // per-task
    struct file *ofile[NOFILE]; // per-task (POSIX와 달리 thread도 별도 보유함)
    struct inode *cwd;        // per-task
    char name[16];
```

```
// === thread group ===
struct proc *group_leader; // 자기 자신이거나 leader proc
int tgid; // == leader->pid
uint64 tf_va; // 이 task의 trapframe이 매핑된 user VA
void *ustack; // thread 전용 user stack 베이스 (join에서 사용)
};
```

- 핵심은 공유 자원과 per-task 자원이 명확히 분리되었다는 것이다
 - `tgid` (thread group id)는 “이 thread가 속한 process가 무엇인가”를 식별할 수 있게 해 주며, 이는 `getpid()` 가 per-task PID를 반환하는 것과 대조된다
 - `tf_va` 는 한 page table 안에 여러 trapframe이 매핑되는 환경에서 “내 trapframe이 어디에 매핑 돼 있는가”를 기록하기 위한 필드이다
 - `ustack` 은 thread library가 `malloc` 으로 잡은 user stack을 `kjoin()` 할 때 회수하기 위해 보존된다
 - `ustack` 이 없으면 `thread_join` 시점에 “이 thread가 사용하던 stack이 heap의 어느 주소였는지”를 알 수 없어서 `free` 를 호출할 수 없다
 - 즉, kernel이 stack 베이스를 대신 기억해주는 역할을 하는 것이 `ustack` 이다

1.3 kernel/proc.c - mm_struct API 스킴레톤

```
struct mm_struct *
mm_alloc(void)
{
    struct mm_struct *mm = (struct mm_struct *)kalloc();
    if (mm == 0) return 0;
    mm->pagetable = 0;
    mm->sz = 0;
    mm->refcount = 1; // 생성 시점에 1
    initlock(&mm->lock, "mm");
    return mm;
}

void mm_get(struct mm_struct *mm) { (void)mm; } //TODO
void mm_put(struct mm_struct *mm) { (void)mm; } //TODO
uint64 find_free_tf_va(pagetable_t pagetable) { (void)pagetable; return 0; } // TODO
```

- `mm_alloc()` 만이 완성되어 있고 `mm_get` / `mm_put` / `find_free_tf_va` 는 구현해야 했다
 - `mm_alloc()` : 새로운 주소 공간을 만들 때 호출(`userinit` 함수와 fork path에서 사용됨)
 - `refcount = 1` 로 시작.
 - `mm_get(mm)` : 새로운 thread가 기존 `mm` 을 공유하기 시작할 때 호출
 - `refcount++` 를 필요로 할 것이다
 - `mm_put(mm)` : thread가 해당 `mm` 을 떠날 때 호출
 - `refcount--` 후 0이 되면 `pagetable` 을 free하고 `mm` 자체도 해제해야함
 - “The last one out clean up”이 적용돼야 한다
 - `find_free_tf_va(pagetable)` : thread 생성 시 `NTHREAD` 개의 trapframe slot 중 비어 있는 슬롯의 VA를 찾아 반환하는 함수
- 부팅 직후 `usertests` 에서 “lost some free pages” 경고가 발생하는 것은 fork test에서 fork-exit cycle마다 `mm` 자체가 해제되지 않기 때문이다
- 즉, `mm_put` 이 비어 있는 한 누적된 누수가 계속된다

1.4 mm 을 참조하는 함수들

- `pagetable` 과 `sz` 가 `mm` 안으로 이동했으므로, 이전에 `p->pagetable`, `p->sz` 로 접근하던 모든 코드가 `p->mm->pagetable`, `p->mm->sz` 로 교체되어야 한다
- 스킴레톤에서는 이미 다음 위치들이 이에 맞추어 수정되어 있다
 - `exec.c` 의 `kexec()`
 - page table 교체 시 `acquire(&p->mm->lock)` 으로 보호된 상태에서 `p->mm->pagetable` 과 `p->mm->sz` 를 갱신한다

- `file.c`, `pipe.c`, `syscall.c`, `vm.c`
 - `copyin` / `copyout`, `fetchaddr` 등 user memory에 접근하는 모든 경로가 `p->mm->pagetable` 을 사용한다
- `sysproc.c::sys_sbrk()`
 - `mm->lock` 을 잡고 `mm->sz` 를 변경한다
- `proc.c::growproc()`
 - 마찬가지로 `mm->lock` 을 잡고 `mm->sz` 를 검증·갱신한다
 - 이때 `USERTOP` 을 초과하는 요청은 거부한다(trapframe slot 영역을 침범하지 않도록)
- `proc.c::userinit()`
 - 최초 user process를 만들 때 `mm_alloc()` 을 호출하여 `p->mm` 을 채우고, 그 안에 page table을 할당한다

1.5 allocproc 과 freeproc 의 thread-aware 초기화

```
// allocproc()
p->mm = 0;
p->group_leader = p;    // 단일 task는 자기 자신이 leader
p->tgid = p->pid;
p->tf_va = 0;
p->ustack = 0;

// trapframe은 per-task이므로 여기서 할당
if((p->trapframe = (struct trapframe *)kalloc()) == 0){ ... }
```

- `allocproc` 은 비어있는 process 껍데기(slot)를 제공하는 함수였다
- 새로 잡힌 proc는 일단 "혼자인 process(leader = self)"로 초기화된다
- `kclone` 에서 thread 경로를 탈 때 비로소 `group_leader`, `tgid`, `mm`, `tf_va`, `ustack` 이 leader/parent 값으로 덮어쓰여질 것이다

```
// freeproc()
if (p->trapframe) { kfree(p->trapframe); p->trapframe = 0; }
p->tf_va = 0;
if (p->mm) { mm_put(p->mm); p->mm = 0; }
...
p->group_leader = 0; p->tgid = 0; p->ustack = 0;
```

- `freeproc` 은 `kfree` 로 per-task 자원 (`trapframe`)을 직접 해제하고, 공유 자원인 `mm` 에 대해서는 `mm_put` 만 호출한다
- `mm_put` 안에서 `refcount` 가 0이 되었을 때만 실제 page table 메모리가 풀려야 하며, 마지막 thread가 죽기 전까지는 pagetable 남아 다른 thread가 계속 사용할 수 있도록 해야 한다
- 다만 thread가 죽을 때는 추가로 해당 task만의 trapframe slot 매핑도 page table에서 제거해야 한다
 - 그래야 같은 슬롯을 다른 thread가 재사용할 수 있다
 - 이 부분이 스켈레톤에서는 구현되지 않았다
 - 명세 Step 1의 "per-thread trapframe slot unmap inside freeproc()"에 해당한다

2. Per-thread Trapframe via sscratch

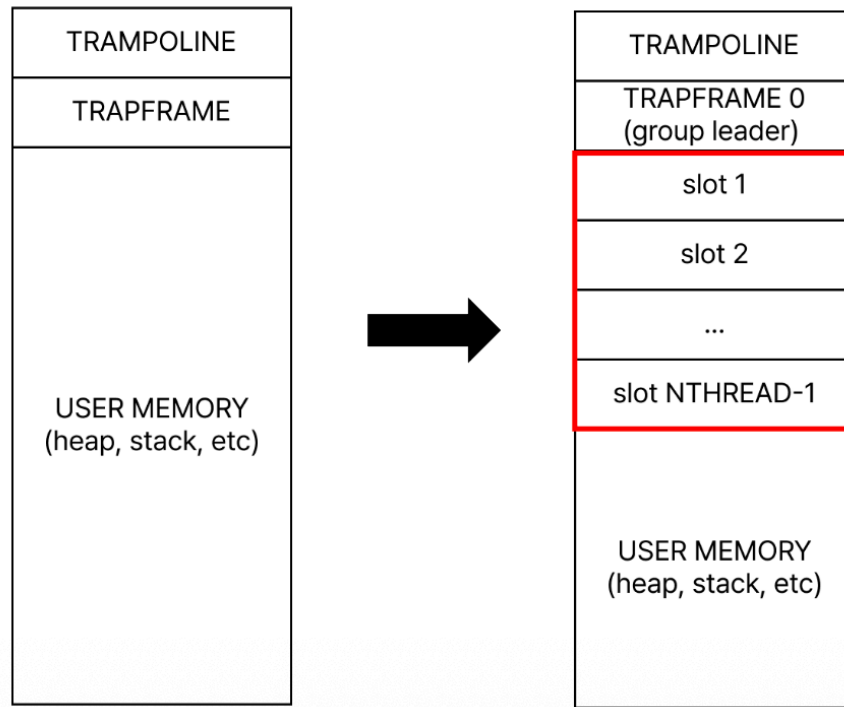
2.1 변경의 동기 분석

- xv6에서 trap이 발생하면 `uservec` (트랩폴린)가 즉시 trapframe에 user register를 저장한다
- 원본 xv6는 trapframe이 모든 process에서 동일한 VA인 `TRAPFRAME` (`TRAMPOLINE - PGSIZE`)에 매핑되므로, 트랩폴린이 `li a0, TRAPFRAME` 처럼 고정 주소를 그냥 사용할 수 있었다
 - 이때 VA는 같아도, PA는 다를 것이다(각자 다른 page table을 사용하므로 문제 없음)
- 그러나 thread는 같은 page table을 공유하므로, "고정된 VA 하나"에 모든 thread가 trapframe을 매핑할 수 없다
- 따라서 충돌하지 않는 **각자의 VA**가 필요하다

2.2 memlayout.h - NTHREAD개의 trapframe slot

```
#define TRAMPOLINE (MAXVA - PGSIZE)
#define TRAPFRAME (TRAMPOLINE - PGSIZE)
#define USERTOP (TRAPFRAME - NTHREAD * PGSIZE)
```

- `param.h`에 `NTHREAD = 8`이 정의되어 있고, 메모리 레이아웃은 다음과 같다.



- `Trapframe`의 값은 변하지 않았으나, 그 의미가 변했다고 볼 수 있다
 - 원래는 모든 process의 trapframe이 매핑되는 유일한 slot
 - 스켈레톤에서는 group leader의 전용 slot(slot 0)
- `USERTOP`이 `TRAPFRAME` 바로 아래가 아니라 `NTHREAD * PGSIZE`만큼 더 아래로 내려간 점이 핵심적인 수정사항이다
 - 원본에서는 trapframe이 딱 한개이므로 `USERTOP`이 `TRAPFRAME` 바로 아래였다
 - 그러므로 원본에서는 `USERTOP`을 따로 정의할 이유가 없었다
 - 그래서 원래 user memory는 `TRAPFRAME` 바로 아래 주소까지만 늘어날 수 있었다(heap의 상한)
- `sbrk` 혹은 `uvmalloc`이 메모리를 늘릴 때 이 영역을 침범하지 못하도록 `growproc()`과 `sys_sbrk()`가 `USERTOP`을 상한으로 검사하는 것은 이 때문이다
 - 만약 이러한 검사가 없다면, heap이 계속 위로 늘어나다가, `trapframe`의 slot을 덮어쓸 수 있다
- Slot 0 = `TRAPFRAME`은 group leader 또는 단일 thread process를 위해 예약되며, `proc_pagetable()`이 leader를 위한 page table을 만들 때 leader의 `trapframe`을 slot 0에 매핑하고 `p->tf_va = TRAPFRAME`을 기록한다

2.3 trampoline.S - sscratch tric

- 원래 xv6 트램폴린은 `li a0, TRAPFRAME`으로 trapframe 주소를 얻었다
- 이는 trapframe VA가 고정된 값(`TRAPFRAME`)이어서 가능했다
- 이제는 thread마다 trapframe VA가 다르므로 "지금 실행중인 thread의 trapframe VA"를 어딘가에 저장했다가 꺼내야 한다
 - 만약 이 경우에도 `li a0, TRAPFRAME`을 사용한다면, 모든 thread가 같은 slot 0에 register를 저장하려고 해서 서로의 상태를 덮어쓸 것이다
- 스켈레톤에서는 다음과 같이 바뀌어 있다

```
uservec:
    # sscratch was set by prepare_return() to the current thread's
```

```

# trapframe virtual address. csrrw atomically swaps a0 with
# sscratch, so afterwards:
#   a0      = trapframe VA (was sscratch)
#   sscratch = user a0      (was a0)
csrrw a0, sscratch, a0
sd ra, 40(a0)
sd sp, 48(a0)
...
csrr t0, sscratch      # 원래 user a0 값을 회수
sd t0, 112(a0)         # trapframe->a0 위치에 저장

```

- `csrrw` 으로 `a0` ↔ `sscratch`를 교환한다는 점을 확인했다
- 이를 통해 다음의 두 가지를 가능하게 했다
 1. `a0` 에는 이 thread의 trapframe VA가 들어옴(prepare_return이 미리 sscratch에 넣어둔 값)
 2. 원래의 user `a0` 값은 `sscratch`로 피신

```

sd ra, 40(a0)
sd sp, 48(a0)
...

```

- 위 코드들은 register 저장되는 것을 보여준다
- 이후 `csrr t0, sscratch`로 회수되어 trapframe의 a0 슬롯(`offset 112`)에 저장된다(user register가 손실되지 않는다)
- 같은 page table 안에 여러 thread의 trapframe이 매핑되어 있더라도, “이 thread는 어느 슬롯을 보아야 하는가”라는 정보가 **각 thread별로 다른 sscratch 값**으로 자연스럽게 나뉘어진다
- 이는 Linux가 `tp` (thread pointer)로 per-thread storage를 식별하는 트릭과 같다
- 복귀 경로(`userret`)도 같은 원리로 동작한다

```

userret:
    sfence.vma zero, zero
    csrw satp, a0          # user page table로 전환
    sfence.vma zero, zero
    csrr a0, sscratch      # 이 thread의 trapframe VA를 다시 로드
    ld ra, 40(a0)
    ...
    ld a0, 112(a0)        # 마지막으로 user a0 복원
    sret

```

- `csrr a0, sscratch`는 `sscratch`를 변경하지 않으므로, 다음 user→kernel trap의 `csrrw`도 여전히 올바른 VA를 찾는다

2.4 `trap.c::prepare_return()` - `sscratch` 갱신

```

void prepare_return(void)
{
    struct proc *p = myproc();
    intr_off();
    uint64 trampoline_uservec = TRAMPOLINE + (uservec - trampoline);
    w_stvec(trampoline_uservec);

    p->trapframe->kernel_satp = r_satp();
    p->trapframe->kernel_sp    = p->kstack + PGSIZE;
    p->trapframe->kernel_trap  = (uint64)usertrap;
    p->trapframe->kernel_hartid = r_tp();

    // 이 thread가 user mode로 나가기 직전에, sscratch에 자신의 tf_va를 저장함
    w_sscratch(p->tf_va);
    ...
}

```

- user space로 돌아갈 때마다 `p->tf_va` 가 `sscratch` 에 저장된다
- scheduler가 다른 thread로 context switch 후 그 thread가 user space로 복귀할 때면, 그 thread의 `prepare_return` 이 호출되어 `sscratch` 가 그 thread의 슬롯 VA로 다시 덮어써진다
- 즉 `sscratch` 는 “현재 user에서 동작 중인 thread의 trapframe VA”를 항상 가리키게 된다

2.5 `proc_pagetable` - slot 0 매핑

```
pagetable_t proc_pagetable(struct proc *p)
{
    ...
    if(mappages(pagetable, TRAPFRAME, PGSIZE,
                (uint64)(p->trapframe), PTE_R | PTE_W) < 0){ ... }
    p->tf_va = TRAPFRAME; // slot 0
    return pagetable;
}
```

- `proc_pagetable` 은 새 page table을 만들 때 호출자(현재는 leader)의 trapframe을 slot 0에 매핑하고 `tf_va` 를 기록한다
- 이때, 두 번째, 세 번째 thread는 새 page table을 만들지 않고 leader의 `mm->pagetable` 을 공유하므로, 빈 슬롯(slot 1, 2, ...)에 자기 trapframe을 따로 `mappages` 해 주어야 한다
- 여기서 사용된 `mappages` 함수는 VA를 PA로 매핑하는 역할을 했다
 - VA → PA의 매핑을 page table에 기록하는 함수
- 이 “빈 슬롯 찾기”가 `find_free_tf_va(pagetable)` 의 역할이다
 - 슬롯별로 `walkaddr` 등으로 매핑 여부를 확인하고, 비어 있는 첫 VA를 돌려준다
 - 이에 대한 구현이 필요하다

2.6 fork 경로의 trapframe 처리

- `kclone()` 의 fork branch는 이렇게 끝난다.

```
*(np->trapframe) = *(p->trapframe); // user register 복사
np->trapframe->a0 = 0; // child의 clone()은 0을 리턴
```

- fork된 자식은 별도의 `mm` 을 가지므로 page table을 새로 만들어 slot 0에 자기 trapframe을 매핑한다
- thread 경로는 이와 달리 leader의 page table을 그대로 공유해야 하므로, 빈 슬롯 찾기 메커니즘을 사용하게 된다

3. Unify `fork` into `clone()` and Add Thread Library

3.1 syscall을 unify하는 이유

- kernel이 제공하는 것은 `clone()` 하나뿐이며, `fork()` 와 `pthread_create()` 는 전달되는 flag bit가 다를 뿐인 같은 호출의 library wrapper 함수이다

3.2 `syscall.h` / `syscall.c` / `usys.pl` - 새 syscall 번호

- `syscall.c` 의 `syscalls` 배열에는 `[SYS_fork]` 가 없어지고, `[SYS_clone]` 과 `[SYS_join]` 이 추가되어 있다

```
static uint64 (*syscalls[])(void) = {
    [SYS_clone]    sys_clone,
    [SYS_join]     sys_join,
    [SYS_exit]     sys_exit,
    [SYS_wait]     sys_wait,
    ...
};
```

- “SYS_fork is gone — fork() is now built on top of clone() in user/ulib.c, so the kernel only sees clone().”(주석의 설명)
- 같은 맥락에서 `usys.pl` 도 `entry("fork")` 대신 `entry("clone")` 과 `entry("join")` 을 생성하며, 끝에 “fork() is no longer a syscall — see user/ulib.c for the wrapper” 라는 주석으로 fork가 더이상 syscall이 아님을 밝힌다

3.3 `sysproc.c`

```

uint64 sys_clone(void)
{
    uint64 fn, arg, stack;
    int n_pages, flags;
    argaddr(0, &fn);
    argaddr(1, &arg);
    argaddr(2, &stack);
    argint(3, &n_pages);
    argint(4, &flags);
    return kclone(fn, arg, stack, n_pages, flags);
}

uint64 sys_join(void)
{
    uint64 stack_addr;
    argaddr(0, &stack_addr);
    return kjoin(stack_addr);
}

```

- `kclone` 호출로 5개 인자(`fn`, `arg`, `stack`, `n_pages`, `flags`)가 모두 kernel로 전달된다
- 이때 `flags`의 `CLONE_VM` 비트 유무가 kernel 내부에서 process(fork)와 thread를 구분되는 부분이었다

3.4 `kclone()` - fork 경로만 완성된 스케레톤

```

int kclone(uint64 fn, uint64 arg, uint64 stack, int n_pages, int flags)
{
    ...
    if ((np = allocproc()) == 0) return -1;

    if (flags & CLONE_VM) {
        // === thread path ===
        freeproc(np);
        release(&np->lock);
        return -1;
    } else {
        // === fork path ===
        if ((np->mm = mm_alloc()) == 0) goto fail;
        if ((np->mm->pagetable = proc_pagetable(np)) == 0) goto fail;
        if (uvmcopy(p->mm->pagetable, np->mm->pagetable, p->mm->sz) < 0) goto fail;
        np->mm->sz = p->mm->sz;
        np->group_leader = np;
        np->tgid = np->pid;

        *(np->trapframe) = *(p->trapframe);
        np->trapframe->a0 = 0;
    }

    // === common: files, cwd, name ===
    for (i = 0; i < NOFILE; i++)
        if (p->ofile[i]) np->ofile[i] = filedup(p->ofile[i]);
    np->cwd = idup(p->cwd);
    safestrncpy(np->name, p->name, sizeof(p->name));
    ...
}

```

- 이 구조에서 추후 구현으로 채워야 할 thread 경로의 **shared**와 **copy**의 차이는 다음과 같다

항목	fork (완성)	thread (구현할 부분)
<code>mm</code>	<code>mm_alloc()</code> 으로 새로 생성	parent의 <code>mm</code> 을 그대로 사용, <code>mm_get()</code> 호출
<code>pagetable</code>	<code>proc_pagetable()</code> + <code>uvmcopy()</code>	parent와 공유, 새 page table 만들지 않음

항목	fork (완성)	thread (구현할 부분)
trapframe (per-task page)	allocproc() 이 새로 할당	마찬가지로 새로 할당
trapframe 매핑 위치	slot 0 (TRAPFRAME)	find_free_tf_va() 로 빈 슬롯을 잡아 mappages
group_leader, tgid	self, self->pid	parent의 leader, parent의 tgid
user sp	parent에서 그대로 복사	stack + n_pages*PGSIZE (top of new stack)
user pc (epc)	parent와 동일 (syscall 다음 명령)	fn
user a0	0 (child 리턴값)	arg (fn의 첫 인자)
ustack	0 (사용 안 함)	stack (kjoin 에서 user에게 돌려주기 위해)

3.5 kjoin() - thread 회수

```
int kjoin(uint64 stack_addr) { (void)stack_addr; return -1; }
```

- kjoin 은 의미상 thread group leader가 호출하며, 자기 그룹의 zombie thread 중 하나를 거두어 그 thread의 user stack 베이스 주소를 user에게 돌려주고 proc 슬롯을 free한다
- kwait 이 자식 process(leader)만 거두는 것과 대조된다
- free() 해야 할 user stack의 베이스가 필요하므로 stack_addr 로 copyout 해야 한다.

3.6 user 측 라이브러리 — ulib.c::fork() 와 uthread.c

```
// user/ulib.c
int fork(void)
{
    return clone(0, 0, 0, 0, 0); // CLONE_VM 비트가 꺼진 clone == fork
}
```

- fork 가 kernel syscall에서 사라졌지만 user 입장에서는 여전히 fork() 를 호출할 수 있다
- 인자 0/0/0/0/0은 "fn/arg/stack/n_pages 없음, flags 0(= CLONE_VM off)"으로 사실상 원본 fork 의미를 그대로 재현한다
- uthread.c 는 구현이 필요한 부분이다

```
int thread_create(void (*fn)(void *), void *arg, int n_pages)
{
    (void)fn; (void)arg; (void)n_pages;
    return -1;
}

int thread_join(void) { return -1; }
```

- 명세에 따르면 thread_create 는
 1. malloc(n_pages * PGSIZE) 로 user stack을 확보하고
 2. clone(fn, arg, stack, n_pages, CLONE_VM) 을 호출하여 thread를 생성한다.
- thread_join 은 join(&stack) 을 호출한 뒤 반환된 stack 베이스에 대해 free(stack) 을 호출하여 자원을 회수한다

3.7 process 전체를 다루는 syscall의 의미 변화

- fork / clone 통합이 끝나면, process 전체를 다루는 다른 syscall들의 의미도 전체적으로 달라진다고 생각했다
- 스케레톤에는 이미 일부 hint가 들어 있다.
 - kexit() 내부:

```
wakeup(p->parent);
if (p->group_leader != p)
    wakeup(p->group_leader);
```

- thread가 죽으면 부모뿐 아니라 group leader도 깨운다
 - leader가 kjoin 에서 자고 있을 수 있으므로

- 다만 명세 Step 3에서 요구하는 "leader가 exit하면 sibling을 죽이고 reap한다 / non-leader가 exit하면 자기만 죽는다"는 부분은 구현해야 했다
- `kwait()` 의 주석: "A sibling thread is NOT a child here — for that, see `kjoin()`."
- 이미 현재 코드는 `pp->parent == p` 만 검사하므로, 이후 구현에서는 추가로 `pp->group_leader == pp` (즉 leader인 자식만 카운트) 조건을 더해야 한다
- `kkill()` 은 현재 단일 task의 `pid` 만 표시하지만, 명세는 "kill the entire group"으로 확장할 것을 요구한다
 - 같은 `tgid` 를 갖는 모든 proc에 `killed = 1` 을 세팅하는 형태가 될 것이다
- `kexec()` 는 page table을 교체하기 직전에 sibling thread를 모두 drain해야 한다(같은 `mm` 을 공유하고 있으므로)
 - 또한 leader가 아닌 thread가 exec을 호출하는 것은 금지된다

Code Analysis 요약

- 스켈레톤이 원본 xv6과 달라지는 점을 요약하자면 다음과 같다
 - `mm_struct` 가 도입되어 공유 가능한 주소 공간을 새로 정의
 - `struct proc` 은 process가 아니라 task(thread)
 - `group_leader / tgid / tf_va / ustack` 의 변수 추가
 - `sscratch` 트릭 덕분에 한 page table 안에 NTHREAD개의 trapframe slot을 매핑할 수 있는 메모리 레이아웃(`USERTOP`, `TRAPFRAME`, `NTHREAD`)과 트랩폴린 코드
 - 슬롯 0은 leader, 슬롯 1..NTHREAD-1은 추가 thread를 위해 비어있음
 - `syscall`에서는 `fork` 가 제거되고 `clone / join` 이 등장했다
 - `kclone()` 의 fork 경로는 완성되어 있음
 - `CLONE_VM` 분기와 `kjoin`, 그리고 user 측 `thread_create / thread_join` 이 추가적인 구현 영역이다
 - `mm_struct` 가 공유 가능한 주소 공간을 만들고, `sscratch` 가 그 공유 주소 공간 안에서 thread별 식별을 가능하게 하며, `clone` 이 두가지를 묶어 "공유할 것"과 "복제할 것"을 한 syscall 안에서 선택할 수 있게 한다

Design

- 본 과제의 목표는 xv6에 Linux의 `clone()` 모델을 단순화한 형태로 도입하여 kernel-level thread를 지원하도록 만드는 것이다
- 구현은 명세가 제시한 세 단계로 나누어 진행했다

Step 1: Memory Management 확장

- `mm_struct` 라는 공유 가능한 주소 공간을 reference counting 기반의 `mm_get / mm_put` 을 구현했다
- 또한 thread마다 별도의 trapframe 슬롯이 필요하므로, page table 안에서 비어 있는 슬롯의 가상 주소를 반환하는 `find_free_tf_va` 를 구현했다
- thread가 종료될 때 `freeproc` 이 그 thread의 trapframe 슬롯을 공유 page table에서 정확히 제거해 주어야 누수가 발생하지 않으므로, 이 unmap 단계도 함께 구현했다
- 이때 원칙은 "마지막에 나가는 자가 청소한다(last-out cleanup)"이다.
 - `mm_get` 은 단순히 `refcount++` 를 수행
 - `mm_put` 은 `refcount--` 후 0이 되었을 때만 실제 page table과 mm 구조체를 해제
 - 락은 카운터 갱신 구간에만 잡고, 실제 free 작업은 락을 풀 뒤 수행하여 데드락 가능성을 차단함

Step 2: Core Threading Primitives

- `kclone()` 의 `CLONE_VM` 분기, `kjoin()`, 그리고 user-space wrapper인 `thread_create / thread_join` 을 구현했다
- fork 경로와 thread 경로의 차이를 명확히 분리해서 구현해야 했다
- user stack의 생애주기는 `thread_create / thread_join` 로 관리하고, kernel은 단지 `ustack` 베이스를 보관 및 반환만 한다

Step 3: Process-wide System Call 의미 재정의

- `struct proc` 이 더 이상 process가 아니라 task(thread)가 되었으므로, process 단위로 동작해야 하는 syscall들의 의미를 그룹 단위로 다시 정의했다
 - `kwait`:

- 자식 중 leader인 것만 카운트한다
- thread는 부모의 자식으로 보이지 않아야 한다
- `kexit` :
 - leader가 호출한 경우 sibling thread를 모두 죽이고, reap한 뒤 자신도 ZOMBIE가 된다
 - reap은 ZOMBIE proc에 대해 `freeproc()` 을 호출하는 것을 의미
 - thread가 호출한 경우 자기 자신만 ZOMBIE가 되고 leader는 나중에 `kjoin` 으로 거둔다
- `kkill` :
 - 같은 `tgid` 를 가진 모든 task에 `killed=1` 을 세팅한다
 - SLEEPING 상태인 task는 RUNNABLE로 깨워 죽음을 인지하게 한다.
- `kexec` :
 - non-leader thread가 호출하면 거부한다
 - leader가 호출했더라도 같은 `mm` 을 공유하는 sibling이 남아 있으면 page table을 교체하기 전에 모두 drain한다
- `kexit` 과 `kexec` 의 drain 로직은 동일하다
- `exec.c` 에서는 이 `drain_siblings(p)` 한 줄만 호출한다

Implementation

Step 1: Memory Management

`mm_get()` / `mm_put()` (`kernel/proc.c`)

```
void
mm_get(struct mm_struct *mm)
{
    acquire(&mm->lock);
    if (mm->refcount < 1)
        panic("mm_get");
    mm->refcount++;
    release(&mm->lock);
}

void
mm_put(struct mm_struct *mm)
{
    acquire(&mm->lock);
    if (mm->refcount < 1)
        panic("mm_put");
    mm->refcount--;
    if (mm->refcount != 0) {
        release(&mm->lock);
        return;
    }
    release(&mm->lock);

    // 마지막 참조자였음 → page table 내용물과 mm 자체를 해제
    if (mm->pagetable) {
        proc_freepagetable(mm->pagetable, mm->sz);
    }
    kfree((void *)mm);
}
```

- refcount 감소는 락 안에서, 실제 free는 락 밖에서 수행한다
- `mm->pagetable` 을 먼저 읽어 page table 내용물을 해제한 다음 `mm` 자체를 해제하는 순서를 유의했다
 - 순서가 반대면 해제된 mm의 필드를 참조하게 된다

`find_free_tf_va()` (`kernel/proc.c`)

```
uint64
find_free_tf_va(pagetable_t pagetable)
{
    for (int i = 0; i < NTHREAD; i++) {
        uint64 va = TRAPFRAME - i * PGSIZE;
        if (ismapped(pagetable, va) == 0) {
            return va;
        }
    }
    return 0;
}
```

- slot 0(=TRAPFRAME)부터 slot NTHREAD-1까지 선형 탐색하며 매핑되지 않은 첫 슬롯의 VA를 반환한다
- `ismapped` 가 1이면 이미 사용 중이므로, 0을 반환할 때만 빈 슬롯이다

`freeproc()` 에 per-thread trapframe slot unmap 추가

```
static void freeproc(struct proc *p) {

    if (p->mm && p->mm->pagetable && p->tf_va != 0 && p->tf_va != TRAPFRAME) {

        acquire(&p->mm->lock);
        uvmunmap(p->mm->pagetable, p->tf_va, 1, 0);
        release(&p->mm->lock);

    }

    if (p->trapframe) {
        kfree((void *)p->trapframe);
        p->trapframe = 0;
    }
    p->tf_va = 0;

    if (p->mm) {
        mm_put(p->mm);
        p->mm = 0;
    }

    ... 종료 ...

}
```

- `uvmunmap` 의 마지막 인자 `do_free = 0` 으로 세팅해야 했다
 - trapframe의 physical page는 바로 다음 줄에서 `kfree` 로 직접 해제하므로, unmap 단계에서는 PTE만 지워야 double free를 피할 수 있었다
- 조건 `p->tf_va != TRAPFRAME` 을 통해 leader(slot 0)는 이 분기를 타지 않게 한다
 - leader의 slot 0은 `mm_put` 내부의 `proc_freepagetable` 이 처리한다

Step 2: Core Threading Primitives

- `kclone()` 의 CLONE_VM 분기 (`kernel/proc.c`)

```
if (flags & CLONE_VM) {
    // mm 공유
    np->mm = p->mm;
    mm_get(np->mm);

    acquire(&np->mm->lock);
    // 공유 page table의 빈 trapframe 슬롯에 새 trapframe 매핑
```

```

uint64 tf_va = find_free_tf_va(np->mm->pagetable);
if (tf_va == 0){
    release(&np->mm->lock);
    goto fail;
}if (mappages(np->mm->pagetable, tf_va, PGSIZE,
            (uint64)np->trapframe, PTE_R | PTE_W) < 0){
    release(&np->mm->lock);
    goto fail;
}
np->tf_va = tf_va;
release(&np->mm->lock);

// thread group 메타
np->group_leader = p->group_leader;
np->tgid         = p->tgid;
np->ustack       = (void *)stack;

// user context: parent에서 복사 후 thread 전용 값으로 덮어씀
memset(np->trapframe, 0, sizeof(*np->trapframe));
np->trapframe->epc = fn;
np->trapframe->sp  = stack + (uint64)n_pages * PGSIZE;
np->trapframe->a0  = arg;
}

```

- fork 경로와 비교하면 다음의 차이가 존재한다

1. `mm_alloc` 대신 `mm` 공유
2. `proc_pagetable` 대신 빈 슬롯 매핑
3. `group_leader`를 `p->group_leader`로 설정(p가 thread여도 leader를 정확히 따라감)
4. `epc` / `sp` / `a0`을 thread 전용 값으로 설정

`kjoin()` (`kernel/proc.c`)

```

int
kjoin(uint64 stack_addr)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();

    acquire(&wait_lock);
    for(;;) {
        havekids = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++) {
            if(pp->group_leader != p->group_leader || pp == p)
                continue;
            acquire(&pp->lock);
            havekids = 1;
            if(pp->state == ZOMBIE) {
                pid = pp->pid;
                if(stack_addr != 0) {
                    uint64 ustack_val = (uint64)pp->ustack;
                    if(copyout(p->mm->pagetable, stack_addr,
                            (char *)&ustack_val, sizeof(ustack_val)) < 0) {
                        release(&pp->lock);
                        release(&wait_lock);
                        return -1;
                    }
                }
            }
        }
    }
    freeproc(pp);
}

```

```

        release(&pp->lock);
        release(&wait_lock);
        return pid;
    }
    release(&pp->lock);
}
if(!havekids || killed(p)) {
    release(&wait_lock);
    return -1;
}
sleep(p, &wait_lock);
}
}

```

- `kwait` 을 참고하되, 자식 판별 조건을 `pp->parent == p` 에서 `pp->group_leader == p->group_leader && pp != p` 로 바꾸었다
- user에게 반환하는 값은 exit status가 아닌 `ustack` 베이스 주소이다.

user-space wrapper (`user/uthread.c`)

```

int
thread_create(void (*fn)(void *), void *arg, int n_pages)
{
    void *stack = malloc(n_pages * PGSIZE);
    if (stack == 0) return -1;
    int tid = clone(fn, arg, stack, n_pages, CLONE_VM);
    if (tid < 0) { free(stack); return -1; }
    return tid;
}

int
thread_join(void)
{
    void *stack = 0;
    int tid = join(&stack);
    if (tid < 0) return -1;
    if (stack) free(stack);
    return tid;
}

```

- stack의 할당과 해제는 user space의 코드인 `thread.c` 가 하는 일이고, kernel은 단지 `ustack` 베이스를 보관/반환만 한다
- `CLONE_VM = 0x0100` 은 `user/uthread.h` 에 추가하여 user-space에서도 참조할 수 있게 했다

Step 3: Process-wide System Calls

`kwait` 수정

```

for(pp = proc; pp < &proc[NPROC]; pp++) {
    if (pp->parent != p) continue;
    if (pp->group_leader != pp) continue;    // ← 추가: leader인 자식만 카운트
    // ... 기존 코드 그대로 ...
}

```

- 이 한 줄을 추가하여 thread를 자식으로 잘못 인식하지 않게 된다

`kkill` 그룹 확장

```

int
kkill(int pid)
{
    struct proc *p;
    struct proc *target = 0;
    int tgid = 0;

```

```

// pid에 해당하는 task의 tgid를 얻음
for(p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if(p->pid == pid) {
        target = p;
        tgid = p->tgid;
        release(&p->lock);
        break;
    }
    release(&p->lock);
}
if (target == 0) return -1;

// 같은 tgid의 모든 task에 killed=1, SLEEPING은 깨움
for(p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if (p->tgid == tgid) {
        p->killed = 1;
        if (p->state == SLEEPING)
            p->state = RUNNABLE;
    }
    release(&p->lock);
}
return 0;
}

```

- 우선 단순히 매칭 및 tgid 추출만 한다
- 이후에 그룹 전체에 표시를 남긴다

kexit 의 leader drain 분기

- `acquire(&wait_lock)` 직후, 기존 `reparent` 호출 직전에 다음 블록을 삽입했다

```

if (p->group_leader == p) {
    struct proc *pp;
    // sibling들에게 killed=1, SLEEPING은 깨움
    for(pp = proc; pp < &proc[NPROC]; pp++) {
        if(pp->group_leader == p && pp != p) {
            acquire(&pp->lock);
            pp->killed = 1;
            if(pp->state == SLEEPING) pp->state = RUNNABLE;
            release(&pp->lock);
        }
    }
}
// 모두 ZOMBIE/UNUSED가 될 때까지 reap + sleep
for(;;) {
    int alive = 0;
    for(pp = proc; pp < &proc[NPROC]; pp++) {
        if(pp->group_leader == p && pp != p) {
            acquire(&pp->lock);
            if(pp->state == ZOMBIE) freeproc(pp);
            else if(pp->state != UNUSED) alive = 1;
            release(&pp->lock);
        }
    }
    if(!alive) break;
    sleep(p, &wait_lock);
}
}

```

- sibling 조건 `pp->group_leader == p && pp != p` 덕분에 leader 자신은 이 루프에서 제외되고, 기존 코드 끝부분에서 ZOMBIE가 되어 부모의 `kwait`이 거두어 간다
- thread가 죽으면 기존 `kexit` 끝의 `wakeup(p->group_leader)`가 leader를 깨워 다음 반복으로 진입시킨다

`drain_siblings()` 헬퍼와 `kexec` 수정

- `kexit`의 drain 블록을 그대로 헬퍼로 추출했다

```
void
drain_siblings(struct proc *p)
{
    struct proc *pp;
    acquire(&wait_lock);
    // ... kexit의 drain 로직과 동일 ...
    release(&wait_lock);
}
```

- `defs.h`에 `void drain_siblings(struct proc *);`를 선언하고, `exec.c::kexec`의 시작 부분에 다음을 추가했다

```
int kexec(char *path, char **argv)
{
    struct proc *p = myproc();
    if (p->group_leader != p)
        return -1;
    drain_siblings(p);
    begin_op();
    // ... 기존 코드 ...
}
```

- `exec.c`는 `proc.c`의 static 함수 `freeproc`이나 내부 전역 `wait_lock`, `proc[]`에 접근할 수 없으므로, drain 로직을 헬퍼로 빼는 것이 자연스럽다고 생각했다

Results

빌드 및 부팅

```
$ make qemu
...
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$
```

`make qemu`가 정상적으로 통과하고 sh이 동작한다. `$ ls`, `$ echo hello` 등 기본 명령이 모두 정상이다.

Step 1 검증 — 메모리 누수 없음

```
$ usertests forktest
usertests starting
test forktest: PASS
PASS ALL TESTS
$
```

- `mm_put` 미구현 상태에서 보였던 "lost some free pages" 경고가 사라졌다
- fork-exit 사이클이 반복되어도 free page가 누수되지 않음을 확인했다

Step 2 검증 — thread 생성/조인

- 다음 테스트 프로그램을 `user/threadtest.c`로 작성하고 실행했다

- thread에 대한 create와 join에 대해 검증하는 코드이다

```
void worker(void *arg) {
    printf("thread running, arg=%d\n", (int)(uint64)arg);
    exit(0);
}

int main() {
    int tid = thread_create(worker, (void*)42, 4);
    printf("created thread tid=%d\n", tid);
    thread_join();
    printf("joined thread, done\n");
    exit(0);
}
```

- 실행 결과:

```
$ threadtest
created thread tid=4
thread running, arg=42
joined thread, done
```

- thread가 정상적으로 생성되고, `arg=42`가 user 함수에 정확히 전달되었으며, `thread_join`이 thread를 거두어 stack을 회수했다

Step 3 검증 — 그룹 단위 syscall

```

▼ TERMINAL
inbeomjin@inbeomjin-ui-MacBookAir ~ /project2-Ben9960 11 main ± make qemu
$ threadtest

=== TEST 1. THREAD CREATION/JOIN TEST ===
THREAD CREATED PID : 4
THREAD JOINED PID : 4
TEST 1 PASS

=== TEST 2. MAXIMUM THREAD TEST ===
TEST 2 PASS

=== TEST 3. SHARED MEMORY TEST ===
TEST 3 PASS. SHARED VALUE : 50

=== TEST 4. THREAD CREATION STRESS TEST ===
CREATING THREADS...
100 THREADS CREATED AND JOINED
200 THREADS CREATED AND JOINED
300 THREADS CREATED AND JOINED
400 THREADS CREATED AND JOINED
500 THREADS CREATED AND JOINED
600 THREADS CREATED AND JOINED
700 THREADS CREATED AND JOINED
800 THREADS CREATED AND JOINED
900 THREADS CREATED AND JOINED
1000 THREADS CREATED AND JOINED
TEST 4 PASS.

=== TEST 5. GROUP LEADER EXIT TEST ===
TEST 5 PASS

=== TEST 6. THREAD GROUP EXEC TEST ===
[PASS] THREAD EXEC FAILED AS EXPECTED
[PASS] LEADER EXEC PASSED.
[PASS] EXEC RESOURCE MANAGEMENT

=== TEST 7. KILL TEST ===
[PASS] KILL TEST PASSED

=== TEST 8. WAIT TEST ===
[PASS] SUCCESSFULLY WAITED FOR GROUP LEADER PROCESS

=== TEST 9. CONCURRENCY STRESS TEST ===
[PASS] CONCURRENCY SBRK TEST PASSED. NO KERNEL PANIC.

```



```

inbeomjin@inbeomjin-ui-MacBookAir ~/project2-Ben9960  main ± make qemu
hart 2 starting
init: starting sh
$ threadtest

=== test_basic_create_join ===
PASS

=== test_shared_memory ===
shared_counter=40000
PASS

=== test_pid_difference ===
leader pid=3
thread pid=9 local addr=0x000000000014FEC
PASS

=== test_many_threads ===
round 0
round 1
round 2
round 3
round 4
round 5
round 6
round 7
round 8
round 9
PASS

=== test_trapframe_slot_reuse ===
PASS

=== test_thread_limit ===
create failed at i=7
created=7 threads
PASS

=== test_fd_duplication ===
PASS parent fd usable

=== test_exec_nonleader_fail ===
exec from non-leader ret=-1
PASS

=== test_kill_group ===
PASS

ALL TESTS DONE

```

Troubleshooting

freeproc 의 unmap 조건 중복

```

// 초기 구현
if (p->mm && p->tf_va != TRAPFRAME) {

```

- `p->tf_va != 0` 이라는 조건이 필요하다(아직 매핑되지 않은 task를 보호)
- 또한 `uvmunmap` 의 세 번째 인자가 누락되어 있었는데, 매핑된 페이지 수를 의미하므로 1을 넘겨야 한다

```

// 수정
if (p->mm != 0 && p->tf_va != 0 && p->tf_va != TRAPFRAME) {
    uvmunmap(p->mm->pagetable, p->tf_va, 1, 0);
}

```

kclone 의 mappages 호출 — 인자 누락과 권한 분리

```

// 초기 구현
mappages(tf_va, PGSIZE, (uint64)np->trapframe, PTE_R, PTE_W)

```

- `mappages` 의 시그니처는 `(pagetable, va, size, pa, perm)` 이다
- 첫 번째 인자 `pagetable` 이 누락되었고, 권한이 콤마로 두 인자처럼 넣는 실수를 했다
 - `PTE_R | PTE_W` 로 하나로 합쳐 마지막 인자에 넣어야 한다

group_leader = p vs group_leader = p->group_leader

- thread 경로의 thread group의 값들을 설정할 때, 처음에 `np->group_leader = p` 로 적었는데, 이는 p가 leader일 경우에만 옳다
- p 자체가 thread인 경우(leader가 또 다른 task) 잘못된 leader를 가리키게 된다
- 항상 안전하게 `np->group_leader = p->group_leader` 로 작성해야 한다

kjoin 의 copyout 인자 오류

```
// 초기 구현
copyout(pp->mm->pagetable, ustack_val, (char *)&pp->xstate, sizeof(pp->xstate))
```

- 세 가지 인자를 잘못 설정했다
 - page table은 pp 가 아니라 p (호출자)의 것이어야 user의 stack_addr 포인터가 유효하다
 - 두 번째 인자는 user가 넘긴 stack_addr 이어야 한다
 - 세 번째 인자는 xstate 가 아니라 ustack_val 의 주소여야 한다

```
// 수정
copyout(p->mm->pagetable, stack_addr, (char *)&ustack_val, sizeof(ustack_val))
```

kexec 에서 non-leader 차단 위치 오류

- kexec() 에 drain_siblings(p) 호출만 추가하고 non-leader에 대한 차단을 빠뜨렸다
- 이렇게 되면 thread가 exec 을 호출할 때도 sibling을 drain하려 시도하는 잘못된 동작을 한다
- 명세는 "exec from non-leader is banned"라고 명시하므로, drain_siblings 보다 먼저 leader 여부를 검사해야 한다

```
// 초기 구현 - non-leader도 drain_siblings를 거침
int kexec(char *path, char **argv)
{
    struct proc *p = myproc();
    drain_siblings(p);
    begin_op();
    ...
}

// 수정 - non-leader는 즉시 거부
int kexec(char *path, char **argv)
{
    struct proc *p = myproc();
    if (p->group_leader != p)
        return -1;
    drain_siblings(p);
    begin_op();
    ...
}
```

freeproc 의 tf_va 조건 : test2 의 instruction page fault 직접 원인

- 가장 까다롭다고 느꼈던 부분이다
- threadtest 의 TEST 2 (MAXIMUM THREAD TEST)에서 다음과 같은 트랩이 반복적으로 발생했다

```
usertrap(): unexpected scause 0xc pid=4
sepc=0xa16 stval=0xa16
usertrap(): unexpected scause 0xc pid=6
sepc=0xa stval=0xa
```

- 여러 task가 0xa, 0xa16처럼 user space의 매우 낮은 주소로 점프하면서 instruction page fault를 일으켰다
- 원인은 freeproc 의 조건문이었다

```
// 초기 구현
if (p->mm && p->mm->pagetable && p->tf_va != TRAPFRAME && p->tf_va != TRAPFRAME) {
    uvmunmap(p->mm->pagetable, p->tf_va, 1, 0);
}
```

- 조건이 잘못된 결과 tf_va == 0 인 task에서도 uvmunmap(pagetable, 0, 1, 0) 이 호출되어 user의 0번째 text page가 강제 unmap되었다

- 이 오류가 어떻게 발생하는지 생각해보았다
 1. TEST 2는 thread 7개를 만들고 8번째 `thread_create` 가 반드시 실패해야 하는 시나리오다
 2. 8번째 `kclone` 호출에서 `allocproc` 이 새 `np` 를 잡고 `np->tf_va = 0` 으로 초기화한다
 3. thread 경로 진입: `np->mm = p->mm; mm_get(np->mm);` 까지 수행
 4. `find_free_tf_va` 가 슬롯이 다 차서 0을 반환 → `goto fail`
 5. `freeproc(np)` 호출 (이 시점에 `np->tf_va == 0` 이지만 `0 != TRAPFRAME` 이므로 잘못된 조건이 참이 된다)
 6. `uvmunmap(p->mm->pagetable, 0, 1, 0)` 실행 → 부모와 공유하는 page table의 0번지 PTE가 지워진다
 7. 부모(leader)와 살아남은 sibling thread들이 user code를 fetch하려다(다음에 실행할 명령어를 읽으려다가) 0번 주소에서 instruction page fault를 일으킨다
 - a. `sepc=0xa` 처럼 매우 낮은 PC 값이 그 증거다
- 조건을 다시 작성하면 이 문제를 해결할 수 있었다

```
// 수정 - 두 번째 조건을 TRAPFRAME → 0 으로 교체
if (p->mm && p->mm->pagetable && p->tf_va != 0 && p->tf_va != TRAPFRAME) {
    acquire(&p->mm->lock);
    uvmunmap(p->mm->pagetable, p->tf_va, 1, 0);
    release(&p->mm->lock);
}
```

- 이 한 줄로 TEST 2의 모든 트랩이 사라졌다
- 이 버그가 특히 위험한 이유는 unmap의 부작용이 그 프로세스 자체가 아니라 공유 **page table**을 쓰는 모든 **sibling**에게 즉시 영향을 끼친다는 점이다